

Patti Royale — an in-game commerce store

Product Associate assignment · PlaySuper

Karan Makol · karan.makol@bluestacks.com · August 2026

Live prototype: <https://daenerysdrag.github.io/patti-royale-playsuper/>

Code: <https://github.com/DaenerysDrag/patti-royale-playsuper>

Best viewed on a laptop — the PM console docks on the right and the reasoning callouts sit beside the phone. Works on mobile too.

Thesis

An in-game store that behaves like a shop will fail, because the player did not come to shop. There is no purchase intent to capture. The job is to convert accumulated play into a reason to play again.

In e-commerce, the cart is dead intent. In a game, the cart is a retention loop.

So I did not build a storefront. I built a coin sink that lifts retention without touching IAP.

Assumptions

WHAT I ASSUMED	
Game	<i>Patti Royale</i> — casual trick-taking cards. 1M DAU, India, Android-first. 1.8 sessions/day, 4 matches/session. Baseline ARPPDAU ₹0.80
Player	Zero purchase intent on entry. ~40 seconds of attention between matches. Starts from the assumption that in-game "free rewards" are a scam
Archetypes	Grinder (daily, never pays, 528 coins/day) · Dipper (2–3×/week, churn-risk, 61/day) · Whale (buys IAP, 235/day)
Currency	Win 50 · loss 15 · daily streak 40. Peg: 10 coins = ₹1
Non-goals	Does not touch the existing cosmetic coin sink. Does not touch IAP pricing. Coins never buy in-game power
Who pays	🔑 The brand funds the discount as customer-acquisition cost — not the studio

The consequence of that last row

If the brand funds the discount as CAC, then the constraint on this economy is **brand CAC budget and coupon inventory — not coin supply**.

Coins can be generous. What must be rationed is **conversion**. That is why there is a redemption cap (2/week) and no paywall on earning — and it is why I never modelled coins as a studio cost.

The core question: how should this differ from e-commerce?

E-COMMERCE ASSUMES	IN-GAME REALITY	SO I BUILT
The user arrives with intent	The player came to play; commerce interrupts	The store opens at the post-match moment , never as a nav tab

E-COMMERCE ASSUMES	IN-GAME REALITY	SO I BUILT
Search is the discovery primitive	40-second window; choice paralysis kills it	No search, no filters. 12 items, 3 fixed groups
Price is an economic fact	Coins are printed by the studio and cost the player <i>time</i>	Price shown in effort — "3 more wins" — computed from your earn rate
Not buying is neutral	"You can't afford this" demotivates — damaging the retention this was sold to fix	Zero dead ends. Every locked item shows a ring and a Play button
Infinite catalog wins	Infinite catalog dilutes	12 SKUs, digital only
A cart for multi-item baskets	Single-item impulse only	No cart. Two taps
Currency is fungible cash	A currency that buys real goods will cannibalise IAP	Real-world rewards only. Never in-game power
The store is the destination	Every second in-store costs the studio the session	Exit CTA is " Back to match ", never "Continue shopping"
Returns handle regret	You cannot refund someone's <i>time</i>	Coins refunded if a voucher expires unused. No e-commerce analogue exists
Growth = more traffic	Growth = more reasons to play	Goals — below

Goals — the mechanic the whole thing rests on

Reserve a reward and it becomes a **ring in the game HUD**, price-locked for 7 days. Every hand fills it. At 100% the game tells you.

This inverts the funnel from `browse → buy` into `want → play → afford → buy`. Commerce stops being an interruption and becomes the reason for the next session. It is also the only part of this design that plausibly moves D7 retention — which is what a studio is actually buying.

Two deliberate consequences:

- `goal_created` is the **conversion event that matters**, not `payment_success`. Creating a goal costs the player nothing and costs the brand nothing, and it is what brings them back.
- **A player holding no goal gets one manufactured.** The reward screen surfaces the nearest out-of-reach item rather than a catalog: close enough to want, far enough to play for.

The economy, and one derivation worth your time

Coin cap is a property of the **item**, never of the player: Tier 1 and 2 are fully coin-funded, **Tier 3 caps coins at 40% and the rest is cash** — PlaySuper's model, and the only place the coin↔cash slider appears. One flat rule, identical for everyone. I deliberately rejected loyalty tiers and engagement-based pricing: charging a more-engaged player more for the same item is manipulative, and it reads that way the moment two players compare screenshots.

Back-solving your published +5.2% ARPDAU rather than guessing a redemption rate:

$\text{₹}0.80 \text{ baseline} \times 5.2\% \text{ lift} = \text{₹}0.0416 \text{ incremental per DAU per day}$
 $\text{₹}700 \text{ order} \times 10\% \text{ commission} \times 50\% = \text{₹}35 \text{ studio revenue per redemption}$

$\Rightarrow 0.0416 / 35 = 0.119\% \text{ of DAU must redeem daily}$
 $\Rightarrow \approx 1,189 \text{ redemptions/day at 1M DAU} \Rightarrow \approx \text{₹}5.35\text{M/month of brand-funded discount}$

Only about 1 player in 840 per day. That is a tiny conversion requirement, and it changes the product: optimise for **retention breadth** — many players holding a goal — over **conversion depth**. A held goal costs the brand nothing and returns a player tomorrow; a checkout costs CAC and may not.

It is also why the funnel step I instrument for is nowhere near the payment.

(₹700 AOV, 10% commission, 50% studio share are my assumptions. If commission is 5%, the required redemption rate doubles and the catalog has to get cheaper — that's the first number I'd want from you.)

Event taxonomy and the named leak

store_tile_impression → store_opened → sku_view → goal_created → checkout_started → payment_success

▲
THE LEAK I INSTRUMENT FOR

Every SKU-scoped event carries `coins_short_by` — the one property that separates a *pricing* failure from an *interest* failure:

DISTRIBUTION	DIAGNOSIS	OWNER
mass at 0	They could afford it and didn't want it → interest	Merchandising / brand mix
mass just above 0	Tier boundary misplaced → pricing	Economy
mass far above 0	Wrong shelf for this player → placement	Surfacing

Three diagnoses, three different owners, three different fixes. Nothing else in the taxonomy can tell them apart, which is why it is attached automatically by `track()` rather than left to call sites. **Event names are a closed TypeScript union — an invented name is a compile error**, so the taxonomy is a contract and not a suggestion.

The PM console in the prototype renders this live off the app you are clicking. Nothing is mocked.

The measurement gap — stated honestly

`voucher_delivered` is the **last event this system can observe**. The brand's real conversion happens off-platform, days later, at Swiggy's checkout.

So `payment_success` is not brand revenue. It is a delivered coupon. Any ROI figure quoted to a brand on the strength of it is a proxy and should be labelled as one. Closing it needs unique per-player codes plus a brand-side redemption webhook reconciled on `code_id`. Until that exists three numbers are unknowable — true redemption rate, true CAC per acquired customer, and incremental-vs-cannibalised brand revenue — so the console greys `voucher_redeemed` out rather than inventing a figure. One fake number would discredit everything else on the panel.

What I deliberately did not build

CUT	WHY
Search, filters, categories	Actively harmful in a 40-second window
Multi-item cart	No browsing intent to justify it
Physical goods, address form	A form mid-session is fatal on a phone, and a delivery promise is a trust liability I haven't earned
Loyalty tiers, dynamic pricing	Same item, same price, everyone. See above
Personalised ranking	The shelf order is fixed. Only the <i>effort copy</i> varies per player
Gift cards, wallet top-ups	Cash-equivalent SKUs are arbitrage with a computable margin — what coin farms optimise against. A voucher with a minimum spend at one merchant has no resale value to a bot, so the merchandising constraint <i>is</i> the fraud control
Separate order history	Coin earns, spends and past claims live in one ledger. Players don't think in "orders", they think "what happened to my coins"

Integrity, since real-world value invites farming: daily earn ceiling (900 — ~1.7× a Grinder's normal day, so it binds bots not humans), device attestation before the **first redemption and never before earning** — gate the payout, not the fun — and one-time codes bound to a player ID. None of it is enforced in a client-only prototype, and I'd rather say so than imply otherwise.

Open questions I would ask on day one

- What coin-cap ceiling do brands actually accept?** I used 40% on Tier 3. It sets the whole economy and I'm guessing.
- Who eats fulfilment failure — brand, PlaySuper, or the studio?** A dead coupon damages trust in *the game*, so the studio has the most to lose and the least control. That asymmetry needs an owner before it needs a process.
- Who calibrates the coin sink?** If each studio does it, every partner re-solves the same economy problem badly.
- Is the coin balance portable across partner games?** A cross-game wallet is a real moat and a real liability the moment a studio churns.
- What is the actual commission and studio split?** It determines the required redemption rate, and therefore whether this catalog is priced right at all.

Tools used

TOOL	WHAT FOR
Claude Code (Opus 5)	All of it — architecture, every screen, the trick-taking match logic, the event bus, the economy derivation, these docs
React 18 · TypeScript · Vite · Tailwind · framer-motion · zustand	Stack. No backend; state persists to <code>localStorage</code>
GitHub Pages + Actions	Deploy. Every push to <code>main</code> typechecks, builds and republishes

How I used it matters more than that I used it: I wrote a **7-phase gated brief** first and made it stop at checkpoints for review, so the decisions stayed mine. Twice it flagged that my own numbers broke — the original earn rate made a "this month" reward affordable in five days, and a single set of price bands cannot be "reach today" for both a Grinder and a Dipper. Both flags changed the design. The economy in this submission is the second version, not the first.

Where the reasoning lives

FILE	WHAT'S IN IT
docs/01-assumptions.md	Game, archetypes, non-goals, commercial inputs
docs/02-economy.md	Peg, faucet, tiers, catalog, guardrails, the derivation
docs/03-event-taxonomy.md	The contract. Properties, funnel, the leak, the measurement gap
docs/04-build-spec.md	Stack, screen graph, state shape, PM panel spec
docs/05-integrity-and-abuse.md	Farming, multi-accounting, voucher abuse